

When workflow agents are enough

Introduction

Workflow agents handle most cases

You learned three workflow agents:

```
Python
SequentialAgent # Fixed order execution
ParallelAgent   # Concurrent execution
LoopAgent       # Iterative execution
```

These cover the vast majority of multi-agent workflows. Before considering custom agents, always ask: “Can I solve this with sequential, parallel, or loop?”

When workflow agents are enough

Use workflow agents for:

Need	Solution
Steps in order	SequentialAgent
Independent parallel tasks	ParallelAgent
Repeat until done	LoopAgent with <code>max_iterations</code>
Combine patterns	Nest workflow agents

Example: Complex nested workflow

Python

```
# Even complex workflows can use standard agents
workflow = SequentialAgent(
    sub_agents=[
        intake_agent,
        ParallelAgent(sub_agents=[research_a, research_b]),
        LoopAgent(sub_agents=[check, refine], max_iterations=3),
        output_agent
    ]
)
```

Key point: You can nest workflow agents to create sophisticated pipelines without custom agents.

When you might need custom agents

Rare scenarios requiring custom logic

Custom agents are needed when you require:

1. Conditional skipping

None

```
IF condition THEN skip Agent B and go directly to Agent C
```

Workflow agents run all sub-agents in their pattern—they can't skip based on conditions.

2. Dynamic routing

None

```
IF type == "billing" THEN route to billing_agent
ELSE IF type == "technical" THEN route to tech_agent
ELSE route to general_agent
```

Workflow agents run all sub-agents in their pattern—they can't skip based on conditions.

3. Complex early exit

None

```
IF quality_score > 8 THEN exit loop early
```

While LoopAgent has `max_iterations`, complex exit conditions need custom logic.

Decision framework

None

```
Can I solve this with Sequential/Parallel/Loop?  
└ YES → Use workflow agents (recommended)  
└ NO → Do I need conditional logic?  
    └ Simple if/else → Consider custom agent  
    └ Complex logic → Custom agent (advanced topic)
```

